



Unidad 3

Principios SOLID

Prof: Lic. Rosales Pablo (prosales@unpata.edu.ar)
Materia: Programación Orientada a Objetos
Año: 2026





La pre clase, en dos preguntas

- La clase pasada terminó con un cero, los métodos de **Tienda** que hubo que tocar para sumar **Indumentaria**.
- Al cierre de caja del local le llegan dos pedidos, un archivo para el contador y un correo para el dueño. Se escriben adentro de **Caja**. ¿Qué tiene de malo, si funciona?
- El contador cambia de formato, el dueño prefiere un mensaje, alguien quiere probar la suma. ¿Dónde termina cada uno de esos pedidos?

Los tres terminan en la misma clase, y ninguno tiene que ver con sumar pagos.



El problema: una clase que hace de todo

```
public class Caja {  
  
    public double cerrar(ArrayList<Pago> pagos) {  
        double total = 0;  
        for (Pago pago : pagos) {  
            total += pago.montoAcreditado();  
        }  
        this.guardarEnArchivo(pagos, total);    // el contador  
        this.enviarResumen(total);              // el dueño  
        return total;  
    }  
}
```

No hay un bug. Hay una clase que va a cambiar por motivos que no son suyos.

**Un diseño se juzga por lo que
cuesta el próximo cambio.**

SOLID son cinco maneras de hacer esa misma pregunta.

—



Cinco letras, cinco principios

- **S**, Single Responsibility. Una clase debe tener una sola responsabilidad.
- **O**, Open/Closed. Las entidades de software deben estar abiertas a la extensión, pero cerradas a la modificación.
- **L**, Liskov Substitution. Los objetos de una subclase deben poder sustituir a los de su superclase sin alterar el funcionamiento del programa.
- **I**, Interface Segregation. Ningún cliente debe verse obligado a depender de métodos que no usa.
- **D**, Dependency Inversion. Los módulos de alto nivel no deben depender de los de bajo nivel: ambos deben depender de abstracciones. Las abstracciones no deben depender de los detalles; los detalles, de las abstracciones.



Single Responsibility

Una sola responsabilidad

- **Caja** tiene tres: sumar el cierre, escribir el archivo y mandar el aviso.
- Para contarlas, mirar quién las pide: el local, el contador y el dueño. Tres personas, tres responsabilidades.

Cómo queda

- **Caja** suma y devuelve el total.
- **ArchivoCierre** escribe el archivo.
- **ServicioCorreo** manda el aviso.
- Y **cerrar()** se prueba sin mandar un solo correo.

Una clase, una responsabilidad. Si dos partes de la clase responden a personas distintas, son dos.



Open/Closed

- Abierta a la extensión, cerrada a la modificación. Suena contradictorio, pero ya pasó dos veces.
- **Tienda** recibió a **Indumentaria** sin abrir el archivo donde estaba escrita, y **cerrarCaja** recibiría igual a **PagoTransferencia**.
- Lo que lo hace posible es abstracción más polimorfismo. El método depende de **Pago**, y cada objeto responde por sí mismo.
- La señal de que falta es una cadena de **if** o de **instanceof** preguntando por el tipo.

Extender el sistema es agregar una clase, no editar las existentes.



Liskov Substitution

```
// Si anular() hubiera ido a Pago, PagoEfectivo la hereda:
@Override
public boolean anular() {
    throw new UnsupportedOperationException("El efectivo no se
anula");
}

// y quien recorre pagos empieza a preguntar antes de llamar:
if (!(pago instanceof PagoEfectivo)) {
    pago.anular();
}
```

La subclase tiene que cumplir lo que la superclase promete. Se perdió Open/Closed por romper Liskov Substitution.



Interface Segregation

Una sola interfaz

- `Operacion` con `anular()`, `estaAnulado()` y `exportar()`.
- `PagoEfectivo` se exporta pero no se anula. Para cumplirla, escribe un `anular()` vacío o con un `throw`.
- Otra vez la clase que miente.

Una por capacidad

- `Anulable` y `Exportable`, separadas. Es lo que se hizo en la clase 6.
- Cada clase firma solo lo que puede cumplir.
- En Java: `Comparable`, `Runnable`, `Iterable`, un método cada una.

Nadie debería depender de métodos que no usa. La señal: un método vacío para que la clase compile.



Dependency Inversion

- Queda un cabo suelto. El cierre sigue teniendo que avisar al dueño: lo que salió de **Caja** fue el cómo se manda un correo, no la decisión de avisar.
- Alguien tiene que llamar a **ServicioCorreo** al terminar, y lo más directo es que lo haga **Caja**, con `new ServicioCorreo("smtp.lapampa.com", 587)` adentro.
- **Caja** es una regla del negocio. **ServicioCorreo** es un detalle. La regla depende del detalle, con servidor y puerto incluidos.
- El enunciado: alto nivel y bajo nivel dependen los dos de una abstracción, definida desde quien la necesita.

Para probar `cerrar()` hace falta un servidor de correo. Esa es la señal.



Dependency Inversion

```
public interface Notificador {  
    boolean enviar(String mensaje);  
}  
  
public class Caja {  
    private final Notificador notificador;  
    // lo entrega quien construye la Caja  
    public Caja(Notificador notificador) {  
        this.notificador = notificador;  
    }  
}
```

La interfaz la define la necesidad de *Caja*, no la existencia del servicio de correo.



Dependency Inversion

```
public double cerrar(List<Pago> pagos) {  
    // ... la suma de siempre ...  
    this.notificador.enviar("Cierre del dia: $" + total);  
    return total;  
}  
  
// ServicioCorreo implements Notificador  
// ServicioMensajes implements Notificador  
// NotificadorDePrueba implements Notificador (guarda el ultimo  
mensaje)
```

La regla de negocio no depende del detalle. Las dos dependen de la interfaz.

El detalle se adapta a la regla, y no al revés.



Inversión de qué

- Antes, **Caja** dependía de **ServicioCorreo**, con servidor y puerto adentro.
- Ahora, **Caja** depende de **Notificador**, y **ServicioCorreo** también. El día que el aviso vaya por mensaje, aparece **ServicioMensajes implements Notificador** y **Caja** no se entera.
- Para probar, un **NotificadorDePrueba** que guarda el último mensaje. Sin servidor.
- Lo mismo en chico: **cerrar()** recibe **List<Pago>**, no **ArrayList<Pago>**. A la suma no le importa cómo está guardada la lista. Es el **List<String> nombres = new ArrayList<>()** de la clase 6.
- El **new ServicioCorreo(...)** tiene que estar en algún lado. Está afuera, en **main** o en la parte que arma los objetos al arrancar. A eso se lo llama inyección de dependencias.



Los cinco, juntos

- **Single Responsibility.** ¿Cuántas responsabilidades tiene esta clase? Señal: una clase que suma, escribe y manda correos.
- **Open/Closed.** ¿Qué hay que abrir para agregar un caso? Señal: `if` o `instanceof` por tipo.
- **Liskov Substitution.** ¿La subclase sirve donde se espera la superclase? Señal: un método heredado que lanza "no soportado".
- **Interface Segregation.** ¿Obliga a implementar lo que no se usa? Señal: un método vacío para compilar.
- **Dependency Inversion.** ¿Depende de una interfaz o de un detalle? Señal: un `new` concreto adentro de la clase.



Lo que no son

- No son reglas de sintaxis. El compilador no verifica ninguno.
- No son una lista para aplicar por adelantado en cada clase. Una clase con tres atributos no necesita una interfaz propia ni un constructor con tres dependencias.
- Se aplican cuando el cambio aparece, o cuando es muy probable que aparezca.

El criterio es siempre el costo del cambio, no la cantidad de clases del proyecto.



La actividad de hoy

- Dominio nuevo, el catálogo de una biblioteca. Una sola clase **Publicacion** con un atributo **tipo**, un **if** por cada caso y el JSON armado adentro.
- **Parte 1:** listar qué habría que tocar para sumar una tesis, y ponerle nombre a cada problema.
- **Parte 2:** **Publicacion** abstracta, **Libro** y **Revista**, la interfaz **Prestable** solo para lo que sale de la sala, y un **Exportador** que **Biblioteca** recibe por constructor.
- **Parte 3:** **Tesis** y **ExportadorCsv**. Contar cuántas líneas de **Biblioteca** hubo que cambiar.

La respuesta de la parte 3 vuelve a ser cero. Y **revista.prestar()** ya no devuelve false, no compila.

En el proyecto integrador

Un `Enemigo` abstracto, una interfaz por capacidad y el `new` de los detalles afuera.
